

Inläsning från tangentbordet

Att läsa in från tangentbordet i C är inte lika enkelt som i C++. Man måste utföra noggrannare felkontroller manuellt och i övrigt göra mera arbete. Man kan antingen läsa i data *teckenvis* eller så kan man läsa in mera data på en gång med hjälp av mera avancerade funktioner. Läser man teckenvis kan man använda `getchar()` och läser man mera på en gång kan man använda `scanf()`. De definieras allmänt:

```
#include <stdio.h>

int getchar(void);

int scanf(const char * format, ...);
```

Av dessa är `getchar()` mycket lätt att använda. Den returnerar nästa tecken som skrivits på tangentbordet. Om ingenting finns väntar funktionen på att något skall skrivas och `enter` tryckas. Ifall inget kunde läsas returneras konstanten `EOF`. Denna indikerar att filslutet nåtts och inget kunde läsas, eller ett annat fel inträffat. Vi kan göra ett enkelt program som skriver ut det som skrevs på skärmen:

```
#include <stdio.h>

int main () {
    char c;

    // läs tecken ända tills vi får ett EOF (end of file)
    while ( (c = getchar ()) != EOF ) {
        putchar ( c );
    }
}
```

Programmet kan avbrytas genom att trycka `control` och `d` för att generera en end-of-file åt programmet.

Inläsning radvis från tangentbordet

Vill man läsa en hel rad från tangentbordet istället för enskilda tecken kan man använda funktionen `gets()`, som allmänt ser ut på följande sätt:

```
#include <stdio.h>

char * gets(char * s);
```

Parametern `s` är pekare till en C-sträng, d.v.s. en buffer dit data kan läsas. Data läses till filen tar slut (end-of-file) eller ett radbyte (`\n`) läses. Radbytet ersätts i buffern `s` med ett termineringstecken `\0`. Denna funktion är dock ganska farlig i och med att den inte kontrollerar bufferns längd på något sätt, så om buffern är för liten utför funktionen ett fel. Undvik `gets()` om du inte vet vad du gör!

Formaterad inläsning

Ifall man inte vill läsa in från tangentbordet tecken- eller radvis, utan vill ha mera funktionalitet kan man läsa formaterad data med hjälp av `scanf()`. Den är definierad allmänt tidigare. Man kan tänka sig att `scanf()` är motsatsen till `printf()` och `fprintf()`. Man skriver formateringssträngen på samma sätt, dock en aning mera begränsade möjligheter, och istället för variabler ges pekare till variabler dit data kan skrivas. Så för att läsa in ett heltal följt av ett flyttal kan följande användas:

```
int Heltal;
float Flyttal;

scanf ("%d %f", &Heltal, &Flyttal);
```

Funktionen försöker nu konvertera nästa text som läses in från tangentbordet till ett heltal följt av ett mellanslag och därefter ett flyttal. Om något annat påträffas misslyckas inläsningen. Funktionen `scanf()` returnerar alltid antalet parametrar som kunde läsas in. I fallet ovan returneras två ifall båda talen kunde läsas in, och noll eller ett om något gick fel. Så vi borde egentligen skrivit:

```
int Heltal;
float Flyttal;

if (scanf ("%d %f", &Heltal,
            &Flyttal) ) != 2 ) {
    // inläsningen misslyckades!
    ...
}
```

Alla datatyper som kan skrivas ut med `printf()` kan läsas in med `scanf()`. Man behöver dock inte specificera t.ex. precision och antal decimaler då man läser in flyttal.

Det är svårare att läsa formaterad data med C än med hjälp av streams i C++, eftersom streams tar hand om en hel del extra uppgifter, såsom att läsa in radslut, extra tomma tecken o.s.v.

[Förutgående](#)
Utskrift till filer

[Hem](#)
[Upp](#)

[Nästa](#)
Inläsning från fil